

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 02 · LECTURE NOTES

Divide and Conquer, and Recurrences

Breaking a problem into pieces, and pricing the result.

MODULE OUTCOMES

- Recognise when a problem has exploitable divide-and-conquer structure.
- Solve recurrences by recursion tree, substitution, and the master theorem.
- State the master theorem's three cases and know when it does not apply.
- Analyse mergesort, Karatsuba, Strassen, and closest-pair.
- Explain why the combine step determines whether the technique pays.

1 The technique

Divide and conquer has three steps, and it is worth being clear which one carries the weight.

- 1 Divide** the problem into subproblems of the same kind. Usually trivial: split the array, split the point set, split the digits.
- 2 Conquer** by recursing. Mechanical.
- 3 Combine** the subproblem answers into an answer for the original. *This is the algorithm.*

COMBINE IS THE WHOLE DESIGN PROBLEM

In mergesort, the merge is what makes it work. In closest-pair, the strip scan is the entire intellectual content — and it depends on a geometric fact that is not obvious.

If your combine step costs as much as solving the problem from scratch, the recursion has bought you nothing. When a divide-and-conquer idea fails to pay off, the combine step is almost always why.

2 Recurrences

$$T(n) = a \cdot T(n/b) + f(n)$$

Symbol	Means	Mergesort	Binary search	Strassen
a	Number of subproblems	2	1	7
b	Size shrink factor	2	2	2

Symbol	Means	Mergesort	Binary search	Strassen
$f(n)$	Divide + combine cost	$\Theta(n)$	$\Theta(1)$	$\Theta(n^2)$
$T(n)$	Result	$\Theta(n \log n)$	$\Theta(\log n)$	$\Theta(n^{2.807})$

Note that a and b are independent. Binary search makes one recursive call on half the data ($a = 1$, $b = 2$); mergesort makes two ($a = 2$, $b = 2$). That difference is the whole distance between $\Theta(\log n)$ and $\Theta(n \log n)$.

3 Method 1: the recursion tree

Draw the tree of recursive calls, label each node with its non-recursive work, sum each level, then sum the levels. It is the method to reach for first, because it tells you *why* the answer is what it is.

At depth i	Count	Size each	Work each	Level total
	a_i	n/b_i	$f(n/b_i)$	$a_i \cdot f(n/b_i)$

- Tree depth is $\log_b n$ — how many times you can divide n by b before reaching 1.
- Leaf count is $a^{\log_b n} = n^{\log_b a}$. Worth deriving once; it is the quantity the master theorem compares against.

Now ask one question: as you descend, does the per-level work grow, shrink, or stay constant?

- **Shrinks** going down $\rightarrow$ the root dominates $\rightarrow T(n) = \Theta(f(n))$.
- **Stays level** $\rightarrow$ every level costs the same $\rightarrow$ multiply by the depth $\rightarrow$ an extra log factor.
- **Grows** going down $\rightarrow$ the leaves dominate $\rightarrow T(n) = \Theta(n^{\log_b a})$.

Those three cases *are* the master theorem. Having seen them in a tree, the theorem stops being three arbitrary conditions to memorise.

4 Method 2: the master theorem

For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$, compare $f(n)$ against $n^{\log_b a}$ — the leaf work.

Case	Condition	Result	Reading
1	$f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$	$\Theta(n^{\log_b a})$	Leaves dominate: combine is cheap relative to the recursion.
2	$f(n) = \Theta(n^{\log_b a})$	$\Theta(n^{\log_b a} \log n)$	Balanced: every level costs the same, so pay the depth.
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ and $a \cdot f(n/b) \leq c \cdot f(n)$ for $c < 1$	$\Theta(f(n))$	Root dominates: the top-level combine is essentially the whole cost.

THREE SITUATIONS WHERE IT DOES NOT APPLY

Unequal splits. $T(n) = T(n/3) + T(2n/3) + n$ is not of the form $aT(n/b)$. Draw the tree instead — it gives $\Theta(n \log n)$, with the unbalanced branches making the analysis more interesting than the answer.

The gaps between cases. $T(n) = 2T(n/2) + n/\log n$ has $f(n)$ smaller than n but not *polynomially* smaller, so no ϵ exists and Case 1 does not apply. There is a gap between each pair of cases.

Wrong form entirely. $T(n) = T(n-1) + n$ is not a divide and conquer recurrence. Expand it directly: it telescopes to $\Theta(n^2)$.

5 Method 3: substitution

Guess the answer and prove it by induction. The only fully general method, and the one worth knowing when the others fail.

The characteristic difficulty is that the obvious guess often fails to carry through the induction *even when it is correct*. The standard fix is to **strengthen the hypothesis** — prove something stronger, which paradoxically makes the proof easier because the inductive step then has more to work with.

For $T(n) = 2T(n/2) + 1$, guessing $T(n) \leq cn$ fails: the inductive step gives $2 \cdot c(n/2) + 1 = cn + 1$, which exceeds cn . Guessing the *stronger* $T(n) \leq cn - d$ succeeds: $2(c(n/2) - d) + 1 = cn - 2d + 1 \leq cn - d$ whenever $d \geq 1$.

WHY A STRONGER CLAIM CAN BE EASIER

Induction gives you the hypothesis for smaller inputs and asks you to establish it for n . A stronger hypothesis hands you more at the start of the step — the extra $-d$ is slack you can spend.

This is a general and genuinely non-obvious proof technique, not a trick specific to recurrences. It recurs in Module 09 and throughout CSCE 627.

6 The classics

6.1 Karatsuba multiplication

Multiplying two n -digit numbers naively splits each into halves and forms four products: $T(n) = 4T(n/2) + O(n)$, which is $\Theta(n^2)$. No improvement over long multiplication.

Writing $x = a \cdot 10^{(n/2)} + b$ and $y = c \cdot 10^{(n/2)} + d$, the product needs ac , bd , and $(ad + bc)$. Karatsuba's observation is that the middle term can be recovered from a third product:

$$ad + bc = (a+b)(c+d) - ac - bd$$

So three multiplications suffice, with additions and subtractions that cost only $O(n)$:

$$T(n) = 3T(n/2) + O(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$$

WHY REMOVING ONE CALL CHANGES THE EXPONENT

The exponent is $\log_b a$ — the number of subproblems sits inside a logarithm. Going from 4 subproblems to 3 moves the exponent from $\log_2 4 = 2$ to $\log_2 3 \approx 1.585$.

Strassen's algorithm is the same idea applied to matrix multiplication: eight half-size products reduced to seven by a similar algebraic identity, giving $\Theta(n^{\log_2 7}) = \Theta(n^{2.807})$.

Both are practical only above a crossover size — the constant factors and the numerical stability are worse — but the idea that *algebra can change an exponent* is one of the genuinely surprising results in the subject.

6.2 Closest pair of points

Given n points in the plane, find the closest pair. Brute force is $\Theta(n^2)$. Divide and conquer gives $\Theta(n \log n)$, and the entire difficulty is in the combine.

Split by a vertical line, recurse on each half, and let d be the smaller of the two results. A closer pair, if one exists, must straddle the line, so both of its points lie within distance d of it — in a vertical strip of width $2d$. The strip may contain all n points, so scanning it pairwise would be $\Theta(n^2)$ and the whole approach would collapse.

THE GEOMETRIC FACT THAT SAVES IT

Sort the strip by y . For any point p , only points within d in y can matter, so consider a $d \times 2d$ rectangle.

Each half of that rectangle is a $d/2 \times d$ square... and any two points in the *same* half were already at distance $\geq d$ from each other, since they came from the same recursive call. A square of side $d/2$ cannot contain two points $\geq d$ apart beyond a fixed number.

The conclusion is that at most a constant number of later points — seven suffices — can be within d of p . The inner loop is $O(1)$, the strip scan is $O(n)$, and the recurrence becomes $T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$.

A purely geometric observation is what makes the complexity work. This is the module's best illustration that combine is where the thinking happens.

7 When to reach for it

Signal	Verdict
Subproblems are independent and combine is cheap	Use it. This is the ideal case.
Algebra can reduce the subproblem count	Use it. Karatsuba, Strassen, FFT.
Subproblems <i>overlap</i> — the same subproblem recurs	Do not. You will recompute exponentially. This is the dynamic programming signal (Module 10).
Combine costs as much as solving directly	Do not. The recursion has bought nothing.
You need parallelism or cache efficiency	Consider it anyway. Independent subproblems parallelise trivially, and shrinking subproblems eventually fit in cache without being tuned for it — the basis of cache-oblivious algorithms.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J — Divide and Conquer, Master Theorem	The master theorem derived from the recursion tree rather than stated, plus Strassen and closest-pair.
MIT 6.006 Lecture — Divide and Conquer	Gentler treatment with more worked recurrences.
Jeff Erickson, Algorithms — Chapter 1 (Recursion)	The best written treatment of recursion trees, with unusually good exercises.
Tim Roughgarden — Karatsuba and the Master Method	Part 1 covers both at an unhurried pace, with the intuition for why the three cases exist.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Solve by recursion tree, then check with the master theorem: $T(n) = 3T(n/2) + n$; $T(n) = 2T(n/4) + \sqrt{n}$; $T(n) = 7T(n/3) + n^2$.
- 2 Solve $T(n) = T(n/3) + T(2n/3) + n$ by drawing the tree. The master theorem does not apply. Explain why the answer is $\Theta(n \log n)$ despite the imbalance.
- 3 Prove $T(n) = 2T(n/2) + 1$ is $O(n)$ by substitution. First try the guess $T(n) \leq cn$ and show where it fails; then strengthen to $cn - d$.
- 4 Implement Karatsuba for big integers and measure it against schoolbook multiplication. Find the crossover size and explain why it is not 2.
- 5 Implement closest-pair in $O(n \log n)$. Then deliberately remove the constant bound on the strip scan and measure both versions, confirming the second is quadratic.
- 6 Implement mergesort with a cutoff to insertion sort below size k . Tune k by measurement and explain the optimum in terms of Module 01's discussion of n_0 and constant factors.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Which of the three steps carries the design effort, and why?
- 2 What do a , b , and $f(n)$ represent? Give them for binary search, mergesort, and Strassen.
- 3 Derive the leaf count $n^{\log_b a}$ from the recursion tree.

- 4 State the three master-theorem cases as a single comparison, and give one recurrence for which it does not apply.
- 5 Why can strengthening an induction hypothesis make a proof succeed where the weaker one fails?
- 6 Karatsuba replaces four multiplications with three. Why does that change the *exponent* rather than just the constant?
- 7 What property of a problem tells you divide and conquer is the wrong technique, and what is the right one?